

Using Claude via MCP for TShark / Wireshark

Lab Report — Setting Up and Using an MCP Server to Drive Wireshark/TShark from Claude

Environment: Windows 11

Wireshark / TShark Version: 4.6.8

Python: 3.13 (Windows installer)

MCP Client: Claude Desktop

Interface Layer: Custom MCP server (server.py) exposing Wireshark/TShark tools to Claude

1. Objective

The objective of this lab was to set up an MCP (Model Context Protocol) server that exposes Wireshark's packet capture and analysis capabilities (via the tshark command-line engine) as tools that Claude can call directly. This allows network interface listing, live packet capture, and .pcap file analysis to be driven conversationally through Claude Desktop, instead of using the Wireshark GUI or raw tshark commands manually.

2. Setup and Installation

2.1 Install the Wireshark MCP Server

```
pip install mcp-wireshark
```

2.2 Install Required Python Library

The MCP Python SDK is required for the server to speak the MCP protocol to Claude Desktop:

```
pip install mcp
```

2.3 Install Wireshark

Wireshark (which bundles tshark, dumpcap, and capinfos) was installed from the official installer at [wireshark.org](https://www.wireshark.org), with the Npcap driver option enabled for live packet capture support.

2.4 Configure Claude Desktop

The MCP server was registered with Claude Desktop by adding an entry to `claude_desktop_config.json` (located under `%APPDATA%\Claude\`), pointing to the server script and specifying python as the launch command. Claude Desktop was then fully restarted so it could detect and connect to the new MCP server.

3. Problems Faced and Resolutions

Problem 1: 'tshark' Not Recognized as a Command

After installing Wireshark, running `tshark -v` in Command Prompt returned:

```
'tshark' is not recognized as an internal or external command, operable program or batch file.
```

Cause: Wireshark was installed correctly, but its installation folder was not added to the Windows PATH environment variable, so Command Prompt could not locate tshark.exe by name.

Resolution:

1. Pressed the Windows key, typed “Environment Variables”, and opened Edit the system environment variables.
2. Clicked the Environment Variables... button at the bottom right.
3. Under User variables for om (or System variables), selected Path and clicked Edit.
4. Clicked New and added the path to the Wireshark installation:

```
C:\Program Files\Wireshark
```

5. Saved all dialogs, closed the existing Command Prompt window, and opened a new one (required for PATH changes to take effect).
6. Re-ran tshark -v to confirm — it now printed the TShark version banner correctly.

Problem 2: ModuleNotFoundError for mcp.server.fastmcp

When running the MCP server script, Python raised:

```
ModuleNotFoundError: No module named 'mcp.server.fastmcp'
```

Cause: The installed mcp package had been upgraded to v2.0.0, a major rework of the MCP Python SDK. In v2, the FastMCP class used in v1 was replaced with a new MCPServer class under a different import path, so the original v1-style import no longer worked.

Resolution:

- Identified that the installed mcp package (2.0.0) was the new v2 SDK release with breaking API changes from v1.
- Updated the import from 'from mcp.server.fastmcp import FastMCP' to 'from mcp.server import MCPServer'.
- Updated the server initialization from 'FastMCP("wireshark")' to 'MCPServer("wireshark")'.
- Updated the run call at the bottom of the script from 'mcp.run(transport="stdio")' to 'mcp.run()', which defaults to the stdio transport used by Claude Desktop.

After these changes, the script ran without errors — a stdio MCP server produces no console output and waits silently for a host to connect, which confirmed it was working correctly.

4. Verification

With both issues resolved, the MCP server was confirmed working end-to-end using the following checks:

- Claude Desktop's tools/plugins panel listed “wireshark” as a connected MCP server.
- A prompt asking Claude to list network interfaces returned a valid JSON list of interfaces from the machine.
- Claude was able to load and analyze an existing .pcap file (2026-02-28-traffic-analysis-exercise.pcap) by path, returning protocol hierarchy and conversation statistics pulled directly via tshark.

5. Demonstration: Using the MCP Server

As a practical demonstration, the MCP server was used to analyze an existing capture file (2026-02-28-traffic-analysis-exercise.pcap, 6.58 MB, 15,512 packets) directly through a conversation with Claude. Claude used the connected tools to pull the capture summary, protocol hierarchy, and top TCP conversations without any manual tshark commands being typed.

Capability Demonstrated	MCP Tool Used
List available network interfaces	list_interfaces
Locate a stored capture file	list_capture_files
Get overall capture metadata (duration, size, packet count)	get_capture_summary
Break down traffic by protocol	get_protocol_hierarchy
Identify top talkers/conversations	get_conversations

6. Conclusion

This lab successfully demonstrated how Wireshark's packet capture and analysis capabilities can be exposed to Claude through a custom MCP server, enabling conversational, tool-driven network traffic analysis. Two setup issues were encountered — a missing PATH entry for Wireshark, and a breaking API change in the mcp Python SDK's v2 release — both of which were diagnosed from their error messages and resolved without needing to reinstall any software. Once configured, the integration allowed capture files to be analyzed through natural-language prompts in Claude Desktop.